# Day 3 — Strands, Agent Squad, MCP and AgentCore

Thursday 15 Oct 2026 · AWS AIP-C01 exam task covered: **2.1 Agentic AI and tool integrations (part B)** · Exam: Sat 7 Nov

**Today in one paragraph**
A **framework** (Strands Agents, LangGraph, CrewAI, Agent Squad) decides how the agent thinks; **Amazon Bedrock AgentCore** hosts, connects, secures and watches it — for any framework and any model. Tools are shared through **MCP** (often served by AgentCore Gateway). Use multi-agent designs only when one agent can't cope, and use **Step Functions** when the steps are fixed and need audit, retries or human approval.

## 1. The big picture

```
BUILD the agent (framework):  Strands Agents · LangGraph · CrewAI · Agent Squad
                      |  runs on
OPERATE it (Amazon Bedrock AgentCore):
   Runtime · Gateway (tools as MCP) · Memory · Identity · Observability
   · Code Interpreter · Browser · managed harness
                      |  calls
MODELS:  Amazon Bedrock (Converse API) — or any other provider
```

Exam Task 2.1 explicitly names **Strands Agents, AWS Agent Squad, MCP and Step Functions**.

## 2. Strands Agents (AWS open-source agent SDK)

- **Model-driven:** give it a model, a system prompt and tools; the model plans the loop (ReAct) itself.
- A tool is a Python function decorated with **@tool**; the **docstring becomes the tool description** (so it is a prompt).
- Bedrock by default, other providers supported (e.g. Anthropic API directly); native **MCP** tool support.

| Multi-agent pattern | How it works | Use when |
|---|---|---|
| Agents as tools | Specialist agents wrapped as tools of an orchestrator agent | Clear specialist roles, one coordinator |
| Swarm | Peer agents hand work to each other | Collaborative, exploratory tasks |
| Graph | Agents are nodes with defined edges (conditions, branches) | Structured flow with LLM decisions at each node |
| Workflow | Fixed sequence / dependency graph of tasks | Repeatable pipelines |

## 3. AWS Agent Squad (formerly Multi-Agent Orchestrator)

- Open-source (AWS Labs). An LLM **classifier routes each request to the best specialist agent**.
- Keeps **conversation context across agents**, so follow-ups reach the right one.
- Also offers a **SupervisorAgent** (lead agent delegates to a team in parallel).
- Exam cue: 'many specialists, each request must reach the right one, cheaply' → **router / classifier**.

## 4. MCP — Model Context Protocol

- Open standard for connecting AI apps to tools and data — 'USB-C for AI tools'.
- **Client–server:** the agent (MCP client) connects to MCP servers exposing **tools**, **resources** (data) and **prompts** (templates).

- Transports: **stdio** (local) and **Streamable HTTP** (remote).
- Write a tool once; every MCP-capable agent (Strands, LangGraph, Claude…) can use it.
- Exam cue: 'standardise / reuse tools across teams and frameworks' → **MCP**, often via **AgentCore Gateway**.

## 5. Amazon Bedrock AgentCore services

| Service | What it does | Exam trigger |
|---|---|---|
| Runtime | Serverless hosting for agents from **any framework**; each session isolated in its own microVM; long-running sessions (hours). | 'Host LangGraph/Strands agents, session isolation, no infrastructure' |
| Gateway | Turns **APIs, Lambda and OpenAPI specs into MCP tools** with built-in auth; **semantic tool search** across many tools. | 'Expose existing APIs as tools to many agents', 'hundreds of tools' |
| Memory | **Short-term** (session events, with expiry) and **long-term** strategies: semantic facts, user preferences, summaries — scoped by **namespaces**. | 'Remember user preferences across sessions' |
| Identity | **Inbound** auth (who may call the agent) and **outbound** auth (agent acts on behalf of the user in Google, GitHub, Salesforce…) with a token vault; secrets in Secrets Manager. | 'Use the user's own permissions in a third-party service' |
| Observability | Traces of steps, tool calls, tokens, latency (OpenTelemetry) in **CloudWatch** (GenAI Observability; needs Transaction Search enabled). | 'Debug / monitor agents in production' |
| Code Interpreter | Sandbox to run agent-generated code safely. | 'Run generated code safely' |
| Browser | Managed cloud browser the agent can drive. | 'Navigate websites, fill web forms' |
| Managed harness | Config-only agent definition (model, tools, instructions); AgentCore runs the loop. | Replacement for console-created Classic agents |

## 6. Agents Classic → modern equivalent

| Bedrock Agents Classic | AgentCore / framework equivalent |
|---|---|
| Action group + Lambda | Strands @tool, or an AgentCore **Gateway** target (Lambda / OpenAPI) |
| Return of control | Tool executes in your own application / client code |
| Attached Knowledge Base | Retrieval tool calling the KB **Retrieve** API |
| Session + long-term memory | **AgentCore Memory** (short-term + long-term strategies) |
| Trace | **AgentCore Observability** |
| Multi-agent collaboration (supervisor + collaborators) | Strands multi-agent patterns, or Agent Squad |
| Code Interpreter action group | **AgentCore Code Interpreter** |

**Classic multi-agent collaboration modes: Supervisor** (plans, delegates, combines) and **Supervisor with routing** (simple requests go straight to one collaborator) — the same supervisor-vs-router idea.

## 7. Step Functions for agent workflows

- A ReAct-style loop can be a state machine: model call → Choice (tool wanted?) → Task (run tool) → loop. Visible, auditable, built-in **retries**.

- **Human approval:** a task with **.waitForTaskToken** pauses until someone calls **SendTaskSuccess** / SendTaskFailure with the token.
- **Standard** workflows run up to **1 year** with full execution history (audit). **Express** workflows run up to **5 minutes** — wrong choice for multi-day approvals.
- Optimised integration: **Bedrock InvokeModel** task calls a model directly from the state machine.
- Choose it when steps are mostly fixed and you need audit trails, retries, timeouts or human approval.

## 8. Choosing a multi-agent pattern

| Pattern | Strength | Cost |
|---|---|---|
| Single agent, many tools | Simplest, cheapest — **start here** | Gets confused beyond a few dozen tools |
| Router / classifier | Fast, cheap; one specialist answers | No cross-specialist collaboration |
| Supervisor / agents as tools | Handles complex multi-domain requests | More model calls → latency + cost |
| Swarm | Flexible, emergent collaboration | Hard to predict and debug |
| Graph / workflow | Predictable, auditable | Less flexible |

**Exam rule**
Add agents only when one agent can't cope. 'Most cost-effective' usually means the simplest pattern that works.

## 9. Decision rules

1. Framework-agnostic hosting, session isolation, no servers → **AgentCore Runtime**.
2. Existing APIs / Lambda as tools shared by many agents → **AgentCore Gateway (MCP)**.
3. Hundreds of tools, wrong tool picked, huge prompts → **Gateway semantic tool search**.
4. Agent acts in third-party services as the user → **AgentCore Identity** (outbound OAuth).
5. Remember preferences across sessions → **AgentCore Memory, long-term** (user-preference strategy).
6. Trace steps / tokens / latency in production → **AgentCore Observability** in CloudWatch.
7. Each request to exactly one of many specialists → **router / Agent Squad classifier**.
8. Complex request needs several specialists combined → **supervisor / agents as tools**.
9. Fixed steps + human approval + audit trail → **Step Functions Standard** + **waitForTaskToken**.
10. Reuse tools across frameworks and teams → **MCP**.
11. Run generated code safely → **AgentCore Code Interpreter**; drive websites → **AgentCore Browser**.

## 10. Lab work

| Lab | Status |
|---|---|
| Code lab: Strands single agent + supervisor (agents as tools) | Pending — Bedrock on-demand TPM quota is 0 on the new account. Workaround: run with the Anthropic API directly. |
| Console lab: Gateway + Memory + Identity + Observability + Step Functions approval workflow | Provided (needs no model quota). Run it and record the checklist answers. |

## Code lab — Strands (model setup: pick ONE)

```
pip install strands-agents strands-agents-tools "strands-agents[anthropic]" boto3

# Option 1 — Bedrock (once quota is available)
from strands.models import BedrockModel
model = BedrockModel(model_id="us.amazon.nova-lite-v1:0", region_name="us-east-1")

# Option 2 — Anthropic API directly (key in the ANTHROPIC_API_KEY env var, never in code/chat)
from strands.models.anthropic import AnthropicModel
model = AnthropicModel(model_id="claude-opus-5-5", max_tokens=16000)
```

```
from strands import Agent, tool

ORDERS = {"1001": "SHIPPED", "1002": "PROCESSING", "1003": "DELIVERED"}

@tool
def get_order_status(order_id: str) -> str:
    """Returns the current status of a customer order.

    Args:
        order_id: The order number, e.g. 1002
    """
    return f"Order {order_id}: {ORDERS.get(order_id, 'NOT FOUND')}"

@tool
def cancel_order(order_id: str) -> str:
    """Cancels an order. Only orders in PROCESSING status can be cancelled.

    Args:
        order_id: The order number to cancel
    """
    if ORDERS.get(order_id) == "PROCESSING":
        ORDERS[order_id] = "CANCELLED"
        return f"Order {order_id} cancelled."
    return f"Order {order_id} cannot be cancelled ({ORDERS.get(order_id, 'NOT FOUND')})."

order_agent = Agent(model=model, tools=[get_order_status, cancel_order],
                    system_prompt="You handle order status and cancellations. Never guess a status.")

policy_agent = Agent(model=model, system_prompt=(
    "You answer refund and shipping policy questions. Policy: refunds within 30 days of delivery; "
    "shipping takes 3-5 business days; cancelled orders are refunded in 5 days."))

@tool
def order_specialist(query: str) -> str:
    """Handles anything about a specific order: status checks and cancellations."""
    return str(order_agent(query))

@tool
def policy_specialist(query: str) -> str:
    """Answers general refund, return and shipping policy questions."""
    return str(policy_agent(query))

supervisor = Agent(model=model, tools=[order_specialist, policy_specialist],
                   system_prompt="You are a customer support lead. Delegate to the right specialist "
                                 "and combine their answers into one reply.")

supervisor("Cancel order 1002 and tell me when I'll get my refund.")   # BOTH specialists
supervisor("How long does shipping take?")                             # ONE specialist
```

Watch: the first question calls both specialists, the second only the policy one; the supervisor pattern costs more model calls than a single agent.

### Console lab — no model quota needed

| Part | Steps | What to record |
|---|---|---|
| A. Gateway (30 min) | Lambda **order-tools** → AgentCore **Gateways** → create **support-gateway** (inbound JWT, quick-create Cognito) → add **Lambda target** with an inline tool schema (get_order_status, cancel_order). | MCP endpoint URL; inbound (Cognito JWT) vs outbound (IAM role → Lambda); role permission; how it compares to an action group. |
| B. Memory (15 min) | Create **support-memory**: short-term event expiry (e.g. 30 days) + Semantic, User preferences and Summarization strategies. | What each strategy extracts; namespaces; which strategy stores 'prefers email'. |
| C. Identity (10 min) | Inspect the gateway's workload identity; browse OAuth providers; create an API-key provider **demo-api-key** (dummy value). | Secret stored in Secrets Manager; which flow books into the user's own calendar. |
| D. Observability (10 min) | CloudWatch → enable **Transaction Search** → open the GenAI Observability dashboard. | What it would show: sessions, traces, tool calls, tokens, latency, errors. |
| E. Step Functions (25 min) | SQS **approval-queue**; state machine: Pass ExtractDocuments → Pass RiskSummary (stand-in for Bedrock InvokeModel) → SQS SendMessage **Wait for callback** with the task token → Choice → Notify. Add a Retry. Run, poll SQS for the token, then in CloudShell: **aws stepfunctions send-task-success --task-token "…" --task-output '{"approved": true}'**. | Execution event history = audit trail; why a workflow beats an agent here. |

Task-token message body — JSONPath: **{"TaskToken.$": "$$.Task.Token", "case": "loan-123"}** · JSONata: **{% {"TaskToken": $states.context.Task.Token, "case": "loan-123"} %}**

**Cleanup**
Delete the gateway + target, the quick-created Cognito user pool, support-memory, demo-api-key, the order-tools Lambda, the state machine and approval-queue. Optionally disable Transaction Search.

## 11. Blockers and learnings

| What happened | Learning |
|---|---|
| Bedrock on-demand TPM quota = 0 on the new account; requests rejected. | Exam Tasks 2.4 / 5.2: quota exhaustion → **ThrottlingException** / rejections. Fixes: **Service Quotas** increase or AWS Support case, **cross-region inference profiles**, another model, retries with exponential backoff. |
| Bedrock Agents Classic unavailable (maintenance mode). | AWS's direction is **AgentCore**; know both vocabularies for the exam. |
| Using Claude via the Anthropic API instead of Bedrock. | Agent concepts are identical; only field names differ (see table below). Never paste API keys into chats or code. |

| Concept | Bedrock Converse API | Anthropic API |
|---|---|---|
| Tool definition | toolSpec + inputSchema.json | name, description, input_schema |
| Model wants a tool | stopReason = "tool_use" | stop_reason = "tool_use" |
| Tool request | toolUse block (toolUseId) | tool_use block (id) |
| Your result | toolResult (toolUseId) | tool_result (tool_use_id) |

## 12. Key takeaways

- Framework thinks, AgentCore operates — any framework, any model.

- Gateway = tools as MCP + auth + semantic tool search. Identity = inbound vs outbound (on behalf of the user).
- Memory = short-term events with expiry + long-term strategies scoped by namespace.
- Router for 'exactly one specialist'; supervisor for 'combine specialists'; single agent first.
- Step Functions Standard + waitForTaskToken for fixed, auditable, human-approved processes.

## 13. Day 3 quiz — answer from memory (answers in the next note)

1. A company has agents built with LangGraph and Strands. It wants to host both without managing servers, with each user's session isolated. Which service?
2. 40 internal REST APIs must become tools for 6 agent teams, with authentication and no per-team glue code. Best approach?
3. A travel assistant must book meetings in each user's own Google Calendar with that user's permissions. Which AgentCore capability handles this?
4. Billing, technical and sales agents; each request must go to exactly one, as cheaply as possible. Supervisor or router? Name an AWS framework that does it.
5. Loan process: extract documents → LLM risk summary → human underwriter approval → notify. Full audit trail and automatic retries required. What runs it, and what feature handles the approval?

**Next: Practice Set #1 (15 questions, Days 1–3), then Day 4 — Security primer (Task 3.2)**
IAM, KMS, VPC endpoints / PrivateLink, Lake Formation, Macie, PII handling, S3 lifecycle and retention.